

UNIT 3, AGENT DEVELOPMENT WITH PYTHON AND FRAMEWORKS

LECTURE 3

Building on Microsoft Agent Framework

Two lectures of foundation, now the payoff. A real agent on the successor framework, the one you would actually ship. Everything you built by hand, assembled in three lines.

SECTION 00

The payoff

You earned this. A real agent, on the real framework.

RECALL

Where the foundation gets you

UNIT 1

You built the loop by hand

Client, tools, loop, memory, guardrails. Forty lines, all yours.

UNIT 3 L1

You met the landscape

The merger, the map, and every framework word mapped to your code.

UNIT 3 L2

You learned the foundation

Semantic Kernel: kernel, plugins, connectors, the base layer.

THE MOMENT IT ALL PAYS OFF

Every hard thing you did by hand becomes a single argument to one function today. And because you did it by hand, you know exactly what that argument does. You are not trusting magic. You are using a tool whose insides you have already seen.

Today

- A tool is just a plain function with a docstring. Simpler than everything before it.
- Building a real, shippable support agent on Microsoft Agent Framework.
- The three arguments that are your whole Unit 1 agent, named and assembled.
- Async, properly: why `agent.run` is awaited, and what that buys you.
- Memory and the seed of multi-agent: session, and an agent as another agent's tool.

HOLD ON TO THIS

Same triage domain, now as a real Agent Framework agent. You will watch it classify a ticket and report an SLA by choosing and calling your tools, with a loop you did not have to write.

SECTION 01

A tool is just a function

The simplest tool model you have seen.

Three tool models, getting simpler each time

UNIT 1, BY HAND

A function AND a schema dict

SEMANTIC KERNEL

A decorated class

AGENT FRAMEWORK

Just a function

NOTICE THE DIRECTION

Each framework asks less of you than the last, while doing more for you. That is what a maturing ecosystem looks like. And underneath, it is still the same idea you built in Unit 1: a function the model can choose to call. Only the ceremony around it shrank.

The whole tool, in one function

```
def get_sla(
    queue: Annotated[str, "The queue name."] -> str:
    """Get the SLA in hours for a support queue."""
    hours = SLA.get(queue.lower())
    return f"The {queue} queue has a {hours} hour SLA."

TRIAGE_TOOLS = [classify_ticket, get_sla, list_queues]
```

READ WHERE THE SCHEMA COMES FROM

The function name becomes the tool name. The docstring becomes the description. The Annotated type becomes the parameter and its description. You write a normal, well documented function, and the framework has everything it needs. Good documentation stopped being politeness and became the interface.

And they are still just functions

```
# no framework, no model, no agent. just call it.  
>>> get_sla("billing")  
"The billing queue has a 24 hour SLA."  
  
>>> classify_ticket("I was charged twice")
```

HOLD ON TO THIS

Because a tool is a plain function, your whole tool suite is testable with ordinary unit tests, no framework and no model. The same offline testability you got from Semantic Kernel plugins, with even less ceremony. Your tools are just Python.

THIS IS WHY THE COURSE ORDER MATTERS

You spent Unit 2 learning to test tools by invariants and fakes. All of that still applies here, unchanged, because the tools are still ordinary functions. The framework did not take your testing skills away. It kept them useful.

SECTION 02

The agent, assembled

Three arguments, and it is your Unit 1 loop.

The whole agent, in three lines

```
client = OpenAIChatClient(model=..., api_key=..., base_url=...)

agent = client.as_agent(
    instructions="You are a support triage assistant...",
    tools=[classify_ticket, get_sla, list_queues])

result = await agent.run("I was double charged, help")
```

THREE ARGUMENTS, AND YOU KNOW ALL THREE

The client is your connector, your lane. The instructions are your system prompt. The tools are your REGISTRY. And `await agent.run` is the entire for-step-in-range loop you wrote in Unit 1, the thinking, the tool calls, the observations, all of it, handled. You did not lose the loop. You stopped having to write it.

The mapping, complete

OpenAIChatClient

→ your get_client from lanes.py

instructions=

→ your system prompt, since Lecture 3

tools=[functions]

→ your REGISTRY, schema read from the function

await agent.run()

→ your entire Unit 1 for-step loop

AgentSession

→ your Session from Unit 2 Lecture 5

middleware

→ your defended call_tool from Unit 2 Lecture 2

agent.as_tool()

→ an agent as another agent's tool: Unit 4

SECTION 03

Async, and what comes next

Why we await, and the seed of multi-agent.

Why agent.run is awaited

UNIT 1 WAS SYNCHRONOUS

One call blocks the world

`client.chat.completions.create(...)` stopped everything until it returned. Fine for learning. But while it waited on the network, your whole program sat idle, doing nothing else.

THE FRAMEWORK IS ASYNC

`await` yields while it waits

`await agent.run(...)` says: pause here until this returns, but let other work run meanwhile. An agent mostly waits on the network, and `async` turns that waiting into time other requests can use.

WHY THIS IS NOT JUST SYNTAX

A synchronous agent handles one request at a time. An async one can have many agents in flight, each yielding while it waits on its own model call. That is the difference between a demo that serves you and a service that serves thousands. `await` is not decoration. It is how agents scale.

An agent can be another agent's tool

```
triage_agent = client.as_agent(instructions=..., tools=...)

manager = client.as_agent(
    instructions="Delegate to your specialist agents.",
    tools=[triage_agent.as_tool(), ...])
```

LOOK AT WHAT THAT MEANS

A tool can be a function, or it can be another whole agent. A manager agent can call a triage agent the same way it calls `get_sla`. This is how multi-agent systems are built, and it is the entire subject of Unit 4. You are seeing the door today. Next unit we walk through it.

Hold on to this

The loop you wrote by hand is now one awaited call. You know every step it takes.

A tool is just a function

Type hints and a docstring. The framework reads the schema.

The agent is three arguments

Client, instructions, tools. Your whole Unit 1 loop, assembled.

await is how agents scale

One process, many waiting agents. Not syntax, mechanism.

An agent can be a tool

`agent.as_tool()` is the door to multi-agent systems, Unit 4.

Before next session

DO

Ship a real agent

Build an Agent Framework agent with three tools of your own, all plain functions. Run it on a real question and watch it choose and call your tools. This is a shippable agent.

DO

Test the tools alone

Write ordinary unit tests for your three tool functions, with no framework and no model. Confirm the whole tool layer is green before the agent ever runs.

THINK

Design a manager

Sketch, in words, a manager agent that delegates to two specialist agents via `as_tool`. You will build this in Unit 4. Naming the pieces now makes that build easier.

ON THE FIRST ONE

This is genuinely a shippable agent, not a toy. The same three lines, pointed at a real model with real tools, is what runs in production today. The distance between your homework and a deployed agent is mostly the reliability work from Unit 2 and the testing coming in Unit 5.

Next session

Unit 3, Lecture 4

Memory, State, and Retrieval

Your agent can act, but it still forgets. Next we give it real memory on real infrastructure: sessions that persist, state that survives, and retrieval augmented generation, the technique that lets an agent search a pile of documents instead of inventing facts. This is the Knowledge panel from the Foundry tour, finally built for real.

COME WITH

Your shippable agent running, your tool tests green, and a sketch of a manager that delegates via `as_tool`.